

Coordinated release strategy for the Michelangelo open source project —
versioning, CI/CD, nightly builds, and changelog automation.

TARGET VERSION TASKS PHASES ARTIFACTS

Motivation Current State Versions Proposal Artifacts Phases & Tasks Milestones Cadence Comparison Setup

? Why This Project



No coordinated releases

Each artifact (Python, Go, UI, Helm, npm) ships independently with its own version. Users can't tell which versions work together.



Missing publication channels

Python wheel is built but never uploaded to PyPI. Helm chart has no OCI push. Community can't `pip install` or `helm install` the project.



No changelog or release notes

CHANGELOG.md is an empty stub. No automated release notes. Users have no way to know what changed between versions.



Non-standard versioning

Tag format `v0.1.0.beta.2` doesn't follow SemVer or PEP 440. Version numbers are fragmented across 6 components.



No nightly builds

Early adopters and CI pipelines have no way to test latest changes. `dev-release.yml` only fires on push to main, not on a schedule.



No quality gates

No release checklist, no RC soak period, no CVE scanning. Releases ship without structured validation.

! Current State — Critical Gaps

GAP	IMPACT	SEVERITY
No PyPI publishing	Python wheel is built but never uploaded — users can't <code>pip install michelangelo</code>	CRITICAL
No Helm chart publishing	No OCI push or chart repository — can't <code>helm install</code> from registry	CRITICAL
No coordinated release	Each artifact ships independently — version compatibility unknown	CRITICAL
No changelog automation	CHANGELOG.md is empty — users don't know what changed	HIGH
Non-standard tag format	<code>v0.1.0.beta.2</code> breaks SemVer and PEP 440 tooling	HIGH
No release branches	All work on <code>main</code> — no stabilization period for releases	HIGH
No nightly builds	No daily artifacts for early adopters or CI integration	HIGH

Version Fragmentation — Before & After

COMPONENT	FILE	CURRENT	PROPOSED
Python SDK	python/pyproject.toml	0.1.1	0.3.0
UI app	website/package.json	0.0.0	0.3.0
npm core	website/packages/core/package.json	0.2.4	0.3.0
npm rpc	website/packages/rpc/package.json	0.1.0	0.3.0
Helm chart	helm/michelangelo/Chart.yaml	0.1.0 / 0.2.1	0.3.0 / 0.3.0
Go module	go.mod	v0 (implicit)	v0.3.0 (tag)
Git tag	—	v0.1.0.beta.2	v0.3.0

✓ Proposal Summary

Unified SemVer

All artifacts share the same Major.Minor version starting at **v0.3.0**. Patch versions may differ for component-specific fixes.

Monthly cadence

Monthly minor releases with a 1-week RC soak period. Patch releases as needed. Nightlies daily at 02:00 UTC.

Release branches

release/vX.Y branches for stabilization. Fixes go to main first, then cherry-pick. Tags only on release branches.

7 artifacts, 1 release

Python (PyPI), Go containers, UI container, npm packages, Helm chart (OCI) — all published from a single coordinated tag.

Automated changelog

git-cliff generates changelogs from conventional commits. GitHub Releases get auto-populated release notes.

Quality gates

10 CI checks required. Release checklist template. CVE scanning on containers. Compatibility matrix across Python/K8s/Node versions.

● Release Artifact Matrix

Python SDK

Wheel + source distribution
pypi.org/project/michelangelo/

Go Server

Container image
ghcr.io/uber/michelangelo/server

UI App

Container image
ghcr.io/uber/michelangelo/ui

npm Core

@michelangelo/core package
npmjs.com/@michelangelo/core

npm RPC

@michelangelo/rpc package
npmjs.com/@michelangelo/rpc

Helm Chart

OCI Helm chart
ghcr.io/uber/michelangelo/chart

Go Binaries

Cross-compiled (Phase 3)
[GitHub Release assets](#)

► Project Phases & Tasks

Phase 1: Foundation

[UP NEXT — 6 TASKS](#)

001	Adopt conventional commits	CONTRIBUTING.md
002	Add cliff.toml for changelog generation	cliff.toml
003	Add PyPI publishing to release workflow	.github/workflows/release.yaml
004	Add Helm OCI chart publishing	.github/workflows/release.yaml
005	Align all version numbers to 0.3.0	pyproject.toml, package.json, Chart.yaml
006	Fix tag format to standard SemVer	docs + CI references

Phase 2: Automation

PLANNED — 7 TASKS

007	Create nightly.yml workflow	.github/workflows/nightly.yml
008	Create changelog.yml workflow	.github/workflows/changelog.yml
009	Add release checklist issue template	.github/ISSUE_TEMPLATE/
010	Add tag-based triggers to existing workflows	dev-release.yml, ui-release.yml
011	Add nightly runtime warning to Python SDK	python/michelangelo/__init__.py
012	Add version-bump script	scripts/version-bump.sh
013	Add nightly integration test workflow	.github/workflows/nightly-integration.y...

Phase 3: Polish

FUTURE — 5 TASKS

014	Add Go binary builds to release	.github/workflows/release.yml
015	Add CVE scanning workflow	.github/workflows/cve-scan.yml
016	Add nightly retention cleanup workflow	.github/workflows/cleanup-nightlies.yml
017	Add compatibility testing matrix	.github/workflows/compat-matrix.yml
018	Evaluate and configure Release Drafter	.github/release-drafter.yml

▮ High-Level Milestones

○ MILESTONE 1

Phase 1 Complete — Foundation

Conventional commits adopted, git-cliff configured, PyPI + Helm OCI publishing added, all versions aligned to 0.3.0, SemVer tag format established. Community can `pip install` and `helm install` from registries.

○ MILESTONE 2

v0.3.0-rc.1 — First Release Candidate

Phase 2 automation in place. Nightly builds running with integration tests validating all 7 artifacts work together. Changelog auto-generated. Release checklist template available. RC published to all channels for community testing. 1-week soak period begins.

○ MILESTONE 3

v0.3.0 — First Coordinated Release

All 7 artifacts published simultaneously from a single tag. GitHub Release with auto-generated notes. Version-bump script streamlines future releases. The project has a professional, predictable release process.

 MILESTONE 4

Phase 3 Complete — Production Ready

Go binaries cross-compiled and attached to releases. CVE scanning on every container build. Nightly retention cleanup running. Compatibility matrix testing across Python 3.9–3.12, K8s 1.28–1.30, Node 18/20.

Proposed Release Cadence

Week 1–3: Development on main (features merge, nightlies publish)

Week 4: Release candidate cut (RC1 tagged from main)

- |— release/v0.Y branch created
- |— RC published to all channels as pre-release
- |— 1-week soak period begins

Week 5: Final release (if RC stable)

- |— v0.Y.0 tagged on release branch
- |— All artifacts published
- |— Release notes & announcements

Industry Comparison

ASPECT	MICHELANGELO	KUBERNETES	MLFLOW	RAY	AIRFLOW
Cadence	Monthly	Quarterly	~Monthly	~Quarterly	~Monthly

ASPECT	MICHELANGELO	KUBERNETES	MLFLOW	RAY	AIRFLOW
Versioning	Unified SemVer	Unified SemVer	Single artifact	Unified SemVer	Single artifact
Nightly	Daily (all)	No	No	Wheels only	No
RC period	1 week	2 weeks	None	1-2 weeks	1-2 weeks
Changelog	git-cliff	Team	Auto (PRs)	Manual	towncrier
LTS	No (pre-1.0)	14 months	No	No	Limited



Required Setup

SECRET / TOOL	PURPOSE	STATUS
PYPI_TOKEN	Publishing Python packages to PyPI	NEEDED
GITHUB_TOKEN	ghcr.io + GitHub Releases	AVAILABLE
NPM_TOKEN	npm package publishing	CONFIGURED
git-cliff	Changelog generation from conventional commits	INSTALL NEEDED

SECRET / TOOL	PURPOSE	STATUS
actionlint	GitHub Actions workflow linting	RECOMMENDED